How the Layout Optimization Algorithms Work

The reference genetic algorithm, the updated genetic algorithm, the systematic search and the hotel layout exploration, each with worked examples

Naimul Haque

Semester project with RNGD

October 2026

What this report is. A step-by-step explanation of the optimization algorithms in the RNGD AI Studio. It covers only the algorithms: what they search over, how one candidate layout is built and scored, and how each search moves from one candidate to the next. Every step is followed by an example with real numbers.

Where the examples come from. All examples were computed by the code itself on one case and written into this report by a script, so they can be reproduced:

- site `fp1` of RNGD's design-optimization prototype (3 levels, 9,640 SF placeable per level, 2 cores) with the prototype's hotel program (72 rooms, 69 of them guest rooms);
- search budget $24 \times 30 = 720$ layouts, random seed 7, no language model anywhere;
- for the hotel sections, the Project Renegade Living + Lodging program (117 keys).

The short version.

1. No algorithm here moves rooms around directly. Each one searches over a short list of numbers (a *genome*), and a fixed procedure (the *decoder*) turns that list into a complete floor plan, which is then scored (the *fitness*).
2. The reference genetic algorithm (GA) and my first attempt let corridor segments sit anywhere. Their plans left guest rooms without windows.
3. The updated decoder only builds plans an architect would accept: straight corridors one room depth from the facade, rooms in modular bays, a window for every guest room. On the example site it places 72 of 72 rooms where the reference placed 61.
4. Two searches drive that decoder: a genetic algorithm with local search, and a deterministic search that enumerates corridor combinations and then improves the best ones move by move. The second is the default.
5. In the hotel mode an outer loop also generates the building shape, the core position and the floor count, and a separate ranking step (Pareto front and TOPSIS) orders the layouts found.

Built 2026-10-11 22:02 from `reports/optimization/examples.json`.

Contents

1 The problem and the running example 3
1.1 What has to be decided . . . 3
1.2 The running example . . . 3
1.3 Why the search does not move rooms directly . . . 4

2 Algorithm A: the reference genetic algorithm 4
2.1 What a genetic algorithm does . . . 4
2.2 What the result files record . . . 4
2.3 What goes wrong, measured . . . 5

3 Algorithm B: my first attempt (free-form corridors) 5
3.1 Genome and decoder . . . 6
3.2 Result and lesson . . . 6

4 The architectural decoder (used by C, D and E) 6
4.1 Step 1: the bay module . . . 7
4.2 Step 2: candidate spines . . . 7
4.3 Step 3: from genome to active spines . . . 8
4.4 Step 4: trimming to the rooms and connecting . . . 8
4.5 Step 5: bays . . . 9
4.6 Step 6: assigning rooms to bays . . . 9
4.7 Step 7: cutting stubs and measuring distances . . . 10
4.8 The decoder as pseudo-code . . . 12

5 The fitness: one number for a layout 12
5.1 Ten objectives . . . 12
5.2 The weighted sum and the exit penalty . . . 12

6 Algorithm C: the updated genetic algorithm 13
6.1 One generation . . . 14
6.2 The starting population . . . 14
6.3 Selection . . . 14
6.4 Crossover . . . 14
6.5 Mutation . . . 15
6.6 The example run . . . 15

7 Algorithm D: the systematic search (default) 16
7.1 Stage 1: enumerate the spine sets . . . 16
7.2 Stage 2: pattern search from the three best . . . 16
7.3 Stage 3: iterated local search . . . 17
7.4 The whole run on the example . . . 17

8 The algorithms side by side 18

9 Algorithm E: exploring hotel layouts (shape, core and floors) 18

9.1 Sizing a plate . . . 19
9.2 Cores and stairs . . . 19
9.3 The three stages . . . 20
9.4 An upper bound per floor . . . 20

10 Choosing among the layouts: Pareto front and TOPSIS **20**
10.1 Pareto front . . . 20
10.2 TOPSIS . . . 20
10.3 Presets . . . 21

11 How good can any algorithm be? Two upper bounds **21**

12 Summary **22**

A Parameters **22**

B Reproducing this report **23**

1 The problem and the running example

1.1 What has to be decided

A site is a stack of levels. Each level is a grid of 1 ft cells, and every cell is one of: *placeable*, *core* (stairs and elevators), *structure* or *outside*. A program lists the rooms to place: a size, a count, a priority and a few flags (needs a window, should be near a core, one per floor).

A layout answers two questions:

- **Where is the corridor?** A set of placeable cells, the same on every level so that the floors stack.
- **Where is each room?** A rectangle on one level, opening onto the corridor, not overlapping anything.

The layout should place as many rooms as possible, give guest rooms a window, keep the corridor short, satisfy the egress rules, and so on. Section 5 turns that list into one number, the fitness F.

1.2 The running example

The example used throughout is site `fp1`: an L-shaped plate of 151 × 100 cells, 3 levels (L1, L2, L3), 9,640 SF placeable per level and 2 cores. The program is `QA/rooms_hotel.json`:

Table 1: The example program. Category, window and near-core flags are inferred from the room name.

Room type	Size, ft	Area, SF	Instances	Priority	Category	Window	Near core
King	25 × 12	300	63	2	guest	yes	no
ADA corner suite	30 × 24	720	3	5	ada	yes	yes
ADA suite	25 × 20	500	3	3	ada	yes	yes
Janitor's closet	25 × 10	250	3	10	service	no	yes

The janitor's closet is one per floor, so the program has 72 room instances in total, 69 of them keys.



Figure 1: The running example, Level 2. Left: the layout delivered by the reference GA. Right: the layout found by the systematic search with the updated decoder. A red dashed outline marks a guest room without a window; blue lines are window walls.

1.3 Why the search does not move rooms directly

The number of ways to put 72 rectangles on a 151 × 100 grid is astronomically large, and almost all of them are useless: rooms overlap, have no door on a corridor, or sit in the middle of the plate. All the algorithms in this report therefore share one structure (Figure 2):

1. the search proposes a **genome**, a short vector of numbers between 0 and 1;
2. a **decoder** turns the genome into a complete layout by fixed rules;
3. the **fitness** scores that layout;
4. the search uses the score to propose the next genome.

The algorithms differ in two places only: what the decoder does (Sections 2, 3 and 4), and how the search proposes the next genome (Sections 6 and 7).

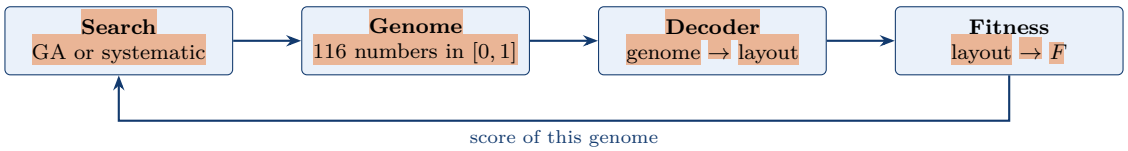


Figure 2: The loop shared by all the algorithms. For the running example the genome has 116 numbers.

Table 2: The algorithms covered in this report.

	Algorithm	Decoder	Search	Same result every run
A	Reference GA (prototype)	free corridor genes	genetic algorithm, 500 generations	not recorded
B	First attempt	free corridor segments, greedy packer	memetic GA	yes (seeded)
C	Updated GA	architectural decoder	memetic GA	yes (seeded)
D	Systematic search (default)	architectural decoder	enumeration, pattern search, iterated local search	yes
E	Hotel exploration	architectural decoder on generated plates	enumeration of plates, pattern search	yes

2 Algorithm A: the reference genetic algorithm

I have the reference GA's *result files* (`QA/floorplan1-3.json`), not its source code. This section describes what those files record and what can be measured from them. It does not guess at internals.

2.1 What a genetic algorithm does

A genetic algorithm keeps a *population* of candidate genomes. In every *generation* it

1. scores every candidate,
2. *selects* better candidates as parents,
3. makes children by *crossover* (taking some genes from one parent and some from the other),
4. *mutates* the children (small random changes),
5. and replaces the population with the children, usually keeping the best old candidates (*elitism*).

Good gene combinations survive and spread; mutation keeps introducing new ones.

2.2 What the result files record

- **A corridor genome.** A list of corridor genes, each three integers. For `fp1` there are 12 genes; the first three are (190, 1, 43), (51, 1, 101), (1994, 3, 105). The corridor they produce is stored as a list of cells (corridor width 5 ft, the same on all levels).

- **Room placements.** For each placed room: level, anchor cell, the side it faces, frontage and depth.
- **A history.** One entry per generation with two numbers: the rooms not yet placed and a score. All three runs have 500 entries.
- **Settings.** A seed, an adjacency weight (0.15) and a balance weight (0.0).

Table 3: The three reference runs, as recorded in their result files.

Site	Genes	Generations	Unplaced, start	Unplaced, end	Last improvement	Placed	Seed
fp1	12	500	36	11	467	61/72	5
fp2	9	500	42	19	407	53/72	6
fp3	7	500	34	15	126	57/72	1

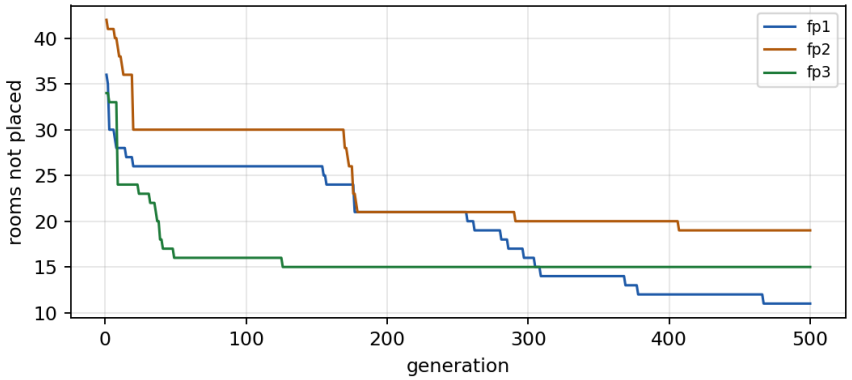


Figure 3: Reference GA: rooms not placed in each of the 500 generations (from the history stored in the result files).

Worked example: reading the history of `fp1`

The run starts with 36 of 72 rooms unplaced and ends with 11 unplaced, so 61 rooms are placed. The last improvement happens in generation 467 of 500: for the remaining 33 generations the best plan does not change.

2.3 What goes wrong, measured

I rebuilt each delivered layout cell by cell and scored it with the fitness of Section 5, the same one used for every other algorithm. For `fp1`:

- 61 of 72 rooms are placed;
- 7 guest rooms have no window, although the file reports an exterior fraction of 1.0 (the two measures differ: mine counts a window only if at least half of the room's back wall lies within 6 ft of the facade);
- 15% of the placeable area is corridor and 16% is neither room nor corridor;
- its fitness is $F = 0.880$.

The cause is the representation. A corridor gene can put a corridor segment anywhere on the plate, and rooms are then attached wherever they touch a corridor. Nothing in the genome knows where the facade is, so whether a room gets a window is left to chance, and the gaps between rooms are too irregular to use. Longer runs do not fix this: the search is looking in a space where almost every candidate is a poor plan.

3 Algorithm B: my first attempt (free-form corridors)

My first optimizer kept the reference idea and improved the parts around it. It is described here because it shows that better scoring and a better search were not enough.

3.1 Genome and decoder

The genome has 10 corridor segments of four genes each, then one *order key* per room:

- **Segment genes** (u, v, o, ℓ). The segment starts at the placeable cell nearest to column $u \cdot (\text{cols} - 1)$ and row $v \cdot (\text{rows} - 1)$. It is horizontal if $o < 0.5$, otherwise vertical. It grows both ways by up to $\ell \cdot \text{span}/2$ cells while the corridor width stays placeable. A segment with $\ell < 0.12$ is switched off.
- **Repair.** All corridor pieces and all cores are joined into one network by least-cost paths.
- **Order keys.** Rooms are placed in decreasing order of $0.06 \cdot \text{priority} + \text{key}$.
- **Greedy packing.** Each room in turn is put at the best free position that touches the corridor. Positions are scored for a window, a corner, distance to a core and how tightly the room sits against its neighbours.

Worked example: decoding one segment

One of the seed genomes has a single active segment $(u, v, o, \ell) = (0.5, 0.81, 0.0, 1.0)$ on the 151×100 grid:

- start near column $0.5 \times 99 \approx 50$ and row $0.81 \times 150 \approx 122$;
- $o = 0.0 < 0.5$, so it runs horizontally;
- reach $= 1.0 \times 100/2 = 50$ cells each way, cut short where the plate ends.

Packing the 72 rooms around this corridor places 50 of them, 3 guest rooms without a window, for a fitness of 0.820 (Figure 4).



Figure 4: The free-form decoder on one seed genome (Level 2).

3.2 Result and lesson

With a memetic GA around this decoder (the same search as Section 6) and the full budget, the example site reached 53 of 72 rooms with 3 windowless guest rooms and $F = 0.880$, in 119 s. Over the three sites it placed 174 rooms against the reference's 171, with 14 windowless guest rooms against 24. The scoring was better and the search was seeded and repeatable, but the plans still had the same faults. The representation had to change.

4 The architectural decoder (used by C, D and E)

The updated decoder builds a floor plan the way a hotel floor is normally laid out. The search no longer decides *where corridor cells are*. It decides *which of a few sensible corridors exist* and *in which order rooms*

claim their places. This section follows one genome through all seven steps: the genome of the best layout the systematic search found for the running example.

4.1 Step 1: the bay module

All rooms are placed on a grid of *bays* along the corridor. One bay is one standard guest-room frontage wide.

- The module M is the **weighted median** of the guest-room frontages, weighted by room count (a suite counts as half its frontage, since it takes two bays).
- D is the median guest-room depth; $D_{\min} = \lfloor 0.92\, d_{\min} \rfloor$ is the shallowest strip a room may use; $D_{\max} = d_{\max} + 2$ the deepest.

Worked example: module of the example program

Guest-room frontages (the shorter side of each room): King 12 ft (×63); ADA corner suite 24 ft (×3); ADA suite 20 ft (×3). Sorted by frontage, the middle one of the 69 guest rooms is a King, so $M = 12$ ft. Depths are 25, 30, 25 ft, so $D = 25$ ft, $D_{\min} = \lfloor 0.92 \times 25 \rfloor = 23$ ft and $D_{\max} = 32$ ft. The corridor is $w = 5$ ft wide. One bay is therefore $12 \times 25 = 300$ SF.

4.2 Step 2: candidate spines

A *spine* is a straight corridor line: a row (horizontal corridor) or a column (vertical corridor). Every row and column k of the plate that can hold a corridor band gets a score

$$s(k) = \sum_{x \in \mathrm{band}(k)} \left[\, g_a(x) + g_b(x) + 1.5\, n_a(x) + 1.5\, n_b(x) + 0.5\, r_a(x) + 0.5\, r_b(x) \,\right].$$

At each foot x along the line, and for each of its two sides a and b:

- $g = 1$ if a room at least $D_{\min}$ deep fits on that side;
- $n = 1$ if, in addition, the facade is about one room depth away (at most $D + 2$+ slack), so a room there has a window;
- $r = 1$ if the facade is within the deepest room's depth.

So one foot of corridor with window rooms on both sides scores $2 \times (1 + 1.5 + 0.5) = 6$, with window rooms on one side only 3, and with deep windowless space on both sides 2.

In each direction the best lines are taken in order of score, skipping any line closer than $D_{\min} + w$ to one already chosen (two corridors that close would leave no room between them). The ten best pieces overall become the candidates. A plate may have fewer than ten.

Worked example: candidate spines of `fp1`

The plate has $m = 5$ candidates (Figure 5, Table 4). Spine 0 is the column one room depth in from the long west facade: it scores 733 over 150 ft, or 4.89 per foot, close to the maximum of 6. The lowest-ranked spine scores 2.00 per foot.

Table 4: Candidate spines of `fp1`, best first.

Spine	Direction	Row / column	Extent	Length, ft	Score s	Score per ft
0	vertical (column)	27	0–149	150	733	4.89
1	horizontal (row)	121	0–99	100	438	4.38
2	vertical (column)	74	95–149	55	196	3.56
3	horizontal (row)	27	25–53	29	116	4.00
4	horizontal (row)	66	0–53	54	108	2.00

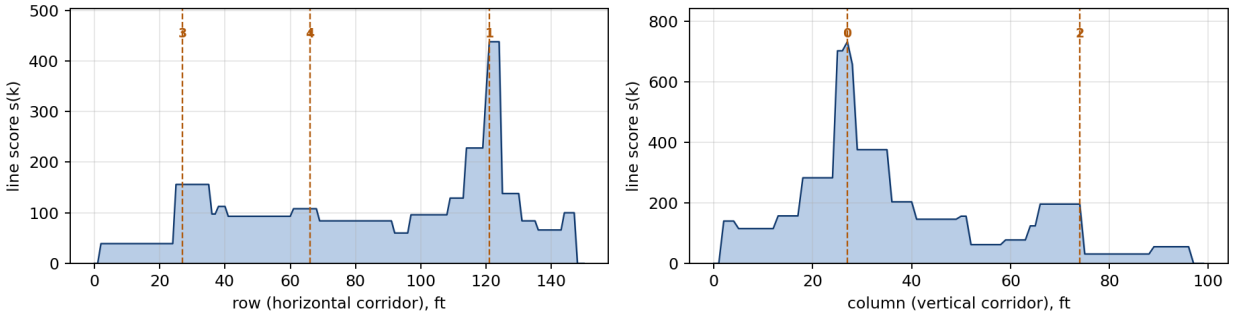


Figure 5: The line score $s(k)$ of every row and every column of `fp1`. Dashed lines mark the candidates chosen (numbered as in Table 4).

4.3 Step 3: from genome to active spines

For 72 rooms the genome has $10 \times 4 + 4 + 72 = 116$ numbers:

- **ten spine blocks** of four genes: trim at the low end, trim at the high end, shift, and an on/off gene (the spine is on if that gene is at least 0.12);
- **four connector genes**, which choose the elbow direction of L-shaped connectors;
- **one order key per room**.

Blocks for candidates the plate does not have are simply ignored (here blocks 5 to 9).

A shift gene h moves the line sideways by $\text{round}(4(h - 0.5))$ ft, so by at most 2 ft. If the shifted line would hit an obstacle, the scored position is kept. A trimmed spine shorter than $2M$ is dropped.

Worked example: the spine genes of the example genome

Spine	Trim low	Trim high	Shift	On/off gene	State
0	0.00	0.00	0.50	1.00	on
1	0.00	0.30	0.50	1.00	on
2	0.00	0.00	0.50	1.00	on
3	0.00	0.00	0.50	1.00	on
4	0.00	0.00	0.50	1.00	on

Spine 1 runs from 0 to 99 ($n = 100$ ft) and has trim genes $(0.00, 0.30)$. The low end moves in by $\lfloor 0.00 \times 0.35 \times 100 \rfloor = 0$ ft and the high end by $\lfloor 0.30 \times 0.35 \times 100 \rfloor = 10$ ft, so it becomes 0–89. A trim gene can remove at most 35% of a spine from each end. All shift genes are 0.5, so no line moves.

4.4 Step 4: trimming to the rooms and connecting

1. A first pass builds bays along every active spine (Step 5) and **cuts each spine back to its own bays**, so no corridor runs past the last possible room. A spine that ends up with no bays of its own is dropped.
2. The remaining spines and all cores are joined into one network. The nearest unconnected piece is linked by an **L-shaped path**: one horizontal and one vertical leg, never a diagonal. A connector gene picks which leg comes first. If both L-shapes are blocked, a rectilinear least-cost path is used.
3. Pieces that still cannot be reached are removed.

Worked example: corridor of the example genome

All 5 spines are switched on, but after the first pass the corridor consists of: column 27, 0–149 (150 ft); row 121, 30–89 (60 ft). The other spines received no bays of their own and were dropped. These spines occupy 1,050 corridor cells and the connectors add 0: here the two spines cross and both cores lie on them, so no connector is needed. The corridor reaches 2 of 2 cores.

4.5 Step 5: bays

Each side of each straight corridor run is cut into bays of width M:

- a run of length ℓ gives $n = \lfloor \ell/M \rfloor$ full bays;
- a remainder $\rho = \ell - nM$ smaller than $0.4M$ is absorbed by the last bay (a wider end bay, which suits a suite); a larger remainder becomes a narrow bay of its own (for a service room);
- a bay's depth is the free depth behind it, capped at $D_{\max}$;
- a bay **has a window** if its back wall is on the facade, and is a **corner** bay if a side wall is too.

Bays are built in **two passes over all corridors**: first every bay that has a window, then windowless bays in whatever space is left. Without this, the windowless inner side of one corridor could take the strip of floor that another corridor needs for its window rooms.

Every level gets the same bays, which is what makes the floors stack.

Worked example: bays of the example layout

There are 27 bays per floor: 27 with a window, 0 without, 3 on a corner. Some of the runs:

- A run of 114 ft facing W: $\lfloor 114/12 \rfloor = 9$ full bays, remainder $6 \geq 4.8$ ft, so it becomes a narrow bay of its own. Bay widths: 12, 12, 12, 12, 12, 12, 12, 12, 12, 6 ft; depth 25 ft.
- A run of 94 ft facing E: $\lfloor 94/12 \rfloor = 7$ full bays, remainder $10 \geq 4.8$ ft, so it becomes a narrow bay of its own. Bay widths: 12, 12, 12, 12, 12, 12, 12, 10 ft; depth 24 ft.
- A run of 60 ft facing N: $\lfloor 60/12 \rfloor = 5$ full bays, no remainder. Bay widths: 12, 12, 12, 12, 12 ft; depth 26 ft.
- A run of 26 ft facing W: $\lfloor 26/12 \rfloor = 2$ full bays, remainder $2 < 0.4 \times 12 = 4.8$ ft, so the last bay absorbs it and is 14 ft wide. Bay widths: 12, 14 ft; depth 25 ft.

4.6 Step 6: assigning rooms to bays

Order. Rooms are taken in decreasing order of

$$0.06 \times \text{priority} + \text{order key}.$$

With all keys equal (0.5), this is pure priority order. The search changes the order by changing keys.

Bays per room. A room of nominal area $\bar{a}$ and frontage f takes

$$k = \max\left(\operatorname{round} \frac{\bar{a}}{MD}, \left\lceil \frac{0.85 f}{M} \right\rceil \right)$$

consecutive bays (the second term applies to guest rooms and makes sure a wide room gets its width).

Table 5: Bays per room type, and the order value with all keys at 0.5 (the janitor's closets go first, the Kings last).

Room type	Area $\bar{a}$	$\bar{a}/(MD)$	by area	Frontage f	by frontage	k	Order value
King	300	1.00	1	12	1	**1**	0.62
ADA corner suite	720	2.40	2	24	2	**2**	0.80
ADA suite	500	1.67	2	20	2	**2**	0.68
Janitor's closet	250	0.83	1	10	–	**1**	1.10

Hard rules. A run of bays is not allowed for a guest room unless

- every bay in the run has a window, and
- the run gives at least 85% of the room's nominal frontage and 85% of its nominal depth.

Score. Among the allowed runs on the allowed levels the room takes the one with the highest score

$$\sigma = W_{\text{ext}}\,[\text{window}] - W_{\text{nw}}\,[\text{no window}] + W_{\text{corner}}\,[\text{corner}] + W_{\text{core}}\left(1 - \tfrac{\min(\delta,150)}{150}\right) - W_{\text{pull}}\,\tfrac{\min(\delta,300)}{300}$$
$$- 1.2\max(0, o - 1.2) + W_{\text{adj}}\,A + W_{\text{stack}}\,\tfrac{\min(S,2)}{2} + W_{\text{grp}}\,[\text{same type next door}] + b_\ell,$$

where δ is the corridor distance from the door to the nearest core, o the ratio of the bays' area to the room's nominal area (oversized rooms waste floor), A the adjacency credit, S the number of other floors that have the

same room type in the same bay (stacking), and b_ℓ a small bonus for a level that has fewer of this room type than average (it spreads rooms over the floors). The window and core terms apply only to rooms that need a window or want to be near a core. A room that does not need a window loses 0.4 for taking a window bay. The weights are $W_{\text{ext}} = 1.0$, $W_{\text{nw}} = 1.5$, $W_{\text{corner}} = 0.8$, $W_{\text{core}} = 1.0$, $W_{\text{pull}} = 0.1$, $W_{\text{adj}} = 0.6$, $W_{\text{stack}} = 0.4$, $W_{\text{grp}} = 0.15$.

Worked example: how many runs are allowed, and how they score

Table 6 counts, for each room type, the runs of k consecutive bays on one floor and how many pass the hard rules. The ADA corner suite is the strictest: it needs 20.4 ft of frontage and 0.85× its depth = 25.5 ft, and only 4 of 22 runs are deep enough.

Table 6: Runs of bays available to each room type on an empty floor.

Room type	k	Runs on a floor	Allowed	Best σ	Worst σ
King	1	27	25	0.998	0.990
ADA corner suite	2	22	4	1.965	1.923
ADA suite	2	22	21	1.965	1.828
Janitor's closet	1	27	26	0.565	0.284

The terms of the score for the ADA suite, for its best and its worst allowed run (on an empty floor, before any room is placed):

Run	Front × depth, ft	o	Oversize	Window	δ, ft	Core	Pull	σ
best	24 × 24	1.15	0.000	+1.000	5	+0.967	-0.002	**1.965**
worst allowed	26 × 25	1.30	-0.120	+1.000	7.4	+0.951	-0.002	**1.828**

The worst run is wider than the room needs ($o = 1.30$, so it loses $1.2 \times (1.30 - 1.2) = 0.12$) and its door is farther from the core.

4.7 Step 7: cutting stubs and measuring distances

After the rooms are assigned, each spine is cut back to what it actually serves: the doors of the placed rooms and its junctions with other corridors and cores. This removes corridor stubs that lead nowhere.

Then two distances are measured along the corridor:

- **Exit travel** T: for each room, the distance from its door to the nearest core plus the room depth; the largest value counts.
- **Dead end** E: the longest distance from any corridor cell to the nearest exit door or to the through-route between two exits. It measures how far someone can walk with only one way out.

Worked example: pruning the example corridor

The corridor shrinks from 1,050 to 970 cells, and the dead end falls from 32.1 ft to 24.7 ft (the limit is 50 ft). The final layout places 72 rooms and leaves 0 unplaced.

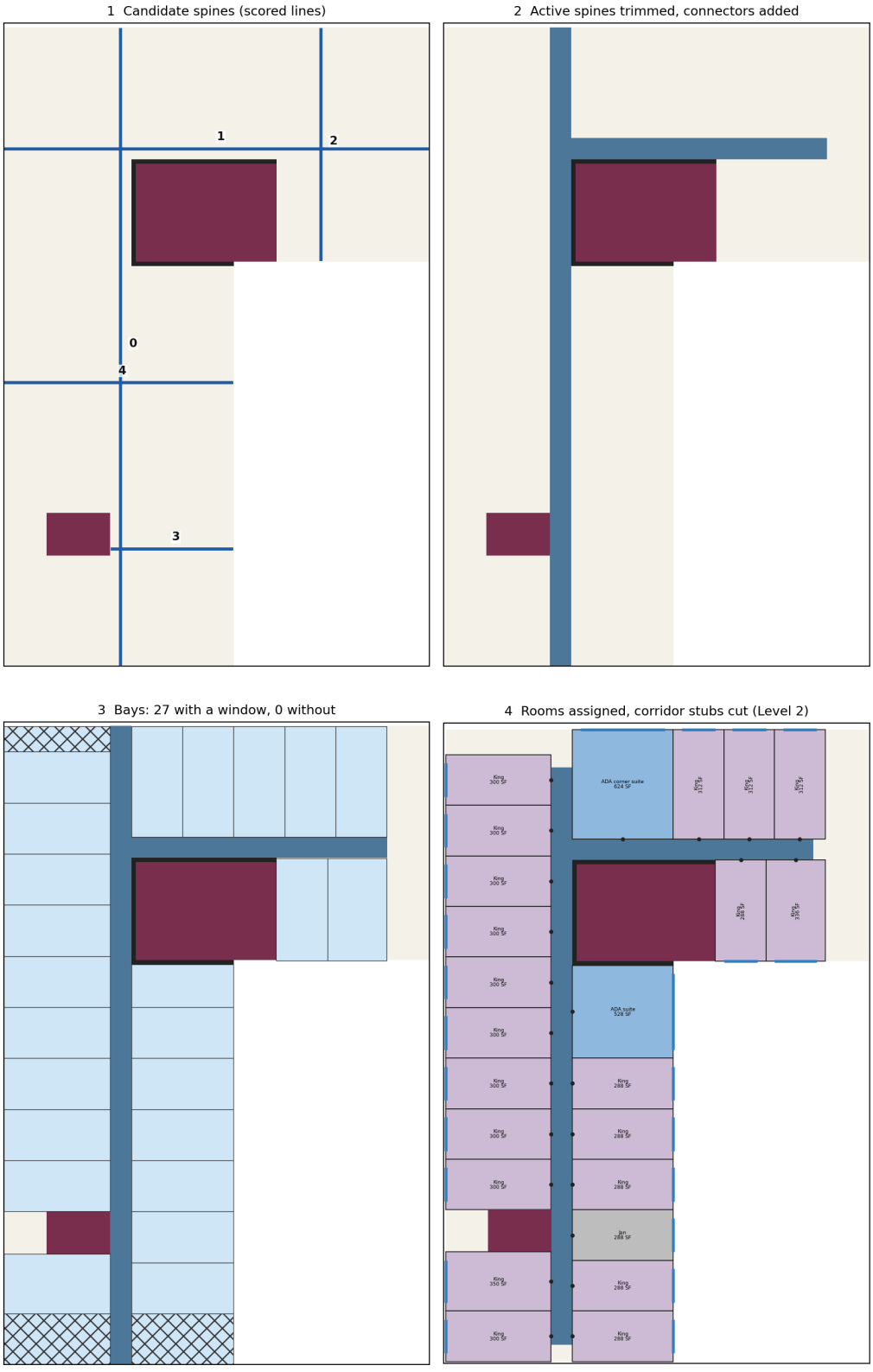


Figure 6: The decoder on the example genome. 1: candidate spines (solid: switched on in this genome). 2: the spines that keep bays of their own, trimmed (blue), with connectors (orange, none needed here). 3: bays (light blue: with a window; hatched: corner). 4: rooms assigned.

4.8 The decoder as pseudo-code

Algorithm 1 Architectural decoder

Require: site, program, genome g

compute M, D, $D_{\min}$, $D_{\max}$ and the candidate spines ▷ once per site and program
$S \leftarrow$ candidates whose on/off gene ≥ 0.12, trimmed and shifted by their genes
build bays along S; cut each spine to its own bays; drop spines with none
corridor $\leftarrow S$ joined to each other and to every core by L-shaped connectors
bays $\leftarrow$ window bays of all corridor sides, then windowless bays in the space left
for each room, in decreasing order of $0.06 \cdot \text{priority} + \text{key}$ **do**
    $k \leftarrow$ bays this room needs
    take the allowed run of k free bays with the highest score σ; if there is none, the room is unplaced
end for
cut each spine back to the doors and junctions it serves
measure exit travel and the dead end along the corridor
return layout

Every genome decodes to a valid plan: corridors are straight, rooms are perpendicular to them, nothing overlaps and every room has a door on the corridor. The search can no longer produce nonsense; it can only choose between plans.

5 The fitness: one number for a layout

The fitness contains no language model. It is arithmetic on the layout's geometry, so the same layout always gets the same score.

5.1 Ten objectives

Each objective f_k is scaled to lie between 0 (worst) and 1 (best):

$$f_{\text{place}} = \frac{\sum_{i \text{ placed}} p_i \min(a_i, \bar{a}_i)}{\sum_i p_i \bar{a}_i}, \qquad f_{\text{day}} = \frac{\#\{\text{rooms needing a window that have one}\}}{\#\{\text{rooms needing a window}\}},$$

$$f_{\text{eff}} = \text{clip}\Big(1 - \frac{\rho}{0.30}\Big), \qquad f_{\text{egress}} = 1 - \tfrac{1}{2}\min\Big(1, \frac{(T - T_{\max})^+}{T_{\max}}\Big) - \tfrac{1}{2}\min\Big(1, \frac{(E - E_{\max})^+}{E_{\max}}\Big),$$

$$f_{\text{ada}} = \min\Big(1, \frac{n_{\text{ADA}}}{n_{224.2}}\Big) \cdot \min\Big(1, \frac{\text{floors with ADA}}{\text{target}}\Big), \qquad f_{\text{core}} = \text{mean}_i\Big(1 - \frac{\min(\delta_i, 120)}{120}\Big),$$

$$f_{\text{adj}} = \frac{\sum_j \omega_j \max\big(0,\ 1 - (\Delta_j - m_j)^+ / m_j\big)}{\sum_j \omega_j}, \qquad f_{\text{corner}} = \frac{\#\{\text{corner rooms on a corner}\}}{\#\{\text{corner rooms}\}},$$

$$f_{\text{bal}} = \text{clip}\Big(1 - \frac{\text{sd}(K_\ell)}{\text{mean}(K_\ell)}\Big), \qquad f_{\text{comp}} = \text{clip}\Big(\frac{2S_{\text{shared}}}{0.6\,P_{\text{rooms}}}\Big).$$

Here p_i is a room's priority, a_i its drawn area and $\bar{a}_i$ its nominal area; ρ is the corridor's share of the placeable area; T and E are the exit travel and dead end of Section 4, with limits $T_{\max} = 250\,\text{ft}$ and $E_{\max} = 50\,\text{ft}$ read from RNGD's IBC rule file; $n_{224.2}$ is the number of accessible rooms required by ADA Table 224.2; δ_i is the door-to-core distance of a room that should be near a core; Δ_j is the distance between the two rooms of a required adjacency and m_j the distance allowed; K_ℓ is the number of keys on guest floor ℓ; S_{shared} is the total length of walls shared by two rooms and P_{rooms} the total room perimeter. $(x)^+$ means $\max(0, x)$, and clip limits a value to $[0, 1]$. An objective with nothing to measure (no required adjacency, no corner room) is 1.

5.2 The weighted sum and the exit penalty

$$F = \frac{\sum_k w_k f_k}{\sum_k w_k} \ - \ \tfrac{1}{2}\max\big(0,\ \min(n_c, 2) - c\big),$$

where n_c is the number of cores on the plate and c the number the corridor reaches. A layout whose corridor misses a required exit loses half the scale, so it can never beat one that reaches both.

Worked example: fitness of the example layout, term by term

Term	Meaning	w_k	f_k	$w_k f_k$	Reference f_k
f_{place}	share of the program placed (priority weighted)	0.30	0.9691	0.2907	0.8911
f_{day}	rooms that need a window have an exterior wall	0.14	1.0000	0.1400	0.8793
f_{eff}	corridor area kept low	0.10	0.6646	0.0665	0.4952
f_{egress}	exit travel and dead-end corridor within IBC limits	0.12	1.0000	0.1200	1.0000
f_{ada}	ADA mobility rooms: count (Table 224.2) and spread over floors	0.08	1.0000	0.0800	1.0000
f_{core}	service and ADA rooms open close to a core/elevator	0.07	0.9683	0.0678	0.9164
f_{adj}	required room adjacencies met	0.07	1.0000	0.0700	1.0000
f_{corner}	suites on corners	0.03	1.0000	0.0300	1.0000
f_{bal}	keys spread evenly over floors	0.05	1.0000	0.0500	0.9512
f_{comp}	rooms share walls (compact, prefab-friendly)	0.04	0.8636	0.0345	0.7036
Sum		1.00		**0.9495**	**0.8798**

- **Placement.** All 72 of 72 rooms are placed, yet $f_{\text{place}} = 0.969$, not 1. Credit is capped at the nominal area, and a few rooms are drawn slightly smaller than nominal (the hard rule allows down to 85%).
- **Efficiency.** The corridor is 970 SF of 9,640 SF placeable, $\rho = 0.1006$, so $f_{\text{eff}} = 1 - 0.1006/0.30 = 0.665$.
- **Egress.** $T = 50.1\,\text{ft} \le 250$ and $E = 24.7\,\text{ft} \le 50$, so nothing is subtracted: $f_{\text{egress}} = 1$. If the dead end were 74 ft instead, the term would be $1 - \frac{1}{2} \times (74 - 50)/50 = 0.76$.
- **ADA.** 69 keys require 4 accessible rooms (Table 224.2). The layout has 6, on 3 floors, so $f_{\text{ada}} = 1$.
- **Core access.** The 9 rooms that should be near a core have their doors 3.8 ft from one on average.
- **Balance.** Keys per floor are 23, 23, 23: standard deviation 0.00, so $f_{\text{bal}} = 1.000$.
- **Compactness.** Shared walls total 1,413 ft and room perimeters 5,454 ft: $2 \times 1,413/5,454/0.6 = 0.864$.
- **Exit penalty.** The corridor reaches 2 of 2 cores, so the penalty is 0.

The weights sum to 1.00, so $F = 0.9495$. The last column scores the reference GA's layout for the same site with the same formulas: it loses most on placement, daylight and corridor efficiency.

Worked example: what the exit penalty does

Take a layout with $F = 0.90$ before the penalty on a plate with two cores. If its corridor reaches only one of them, $F = 0.90 - \frac{1}{2}(2 - 1) = 0.40$. Any layout that connects both exits scores higher, however poor it is otherwise.

6 Algorithm C: the updated genetic algorithm

The updated GA uses the architectural decoder and adds a short local search to every generation, which makes it a *memetic* algorithm. It is seeded, so a run can be repeated exactly.

6.1 One generation

Algorithm 2 Updated (memetic) GA

Require: population size P, generations G, seed
population ← six spine-set seeds + three random genomes + random genomes up to P
for $t = 1$ **to** G **do**
  $\sigma_t \leftarrow \sigma_{\text{start}} + (\sigma_{\text{end}} - \sigma_{\text{start}})\,\frac{t-1}{G-1}$ ▷ mutation strength falls
  next ← the 2 best individuals ▷ elitism
  improve the best one by 4 trial mutations of strength $\sigma_t/2$, keeping each one only if it is better ▷ local search
  if the population's fitness values are nearly equal (standard deviation < 0.01) **then**
    add 2 random genomes ▷ immigrants
  end if
  while next has fewer than P individuals **do**
    pick two parents, each the best of 3 individuals drawn at random ▷ tournament selection
    child ← crossover(parents) with probability 0.9, otherwise a copy of the first parent
    child ← mutate(child, σ_t); decode and score it; add it to next
  end while
  population ← next
end for
return the best individual

6.2 The starting population

A purely random start wastes generations rediscovering the obvious. Six seeds therefore switch on fixed sets of spines: all of them, the best one, the best two, the best three, the best four, and every second one. Three more genomes are random, and the rest of the population is filled with random genomes in which each spine is off with probability 0.4.

6.3 Selection

To pick a parent, 3 individuals are drawn at random and the fittest wins. Better individuals are picked more often, but a weaker one still wins when it happens to be drawn with even weaker ones, which keeps variety in the population.

6.4 Crossover

- For each of the eleven gene blocks (ten spines and the connector block) a coin decides which parent it comes from. A spine is inherited whole: its trims, its shift and its on/off gene stay together.
- For each room's order key another coin decides.

Worked example: a crossover that helps

- Parent 1: spines {0, 1, 2}, 53 rooms, $F = 0.894$.
- Parent 2: spines {0, 2, 4}, 66 rooms, $F = 0.872$.
- The child takes spine blocks {0, 2, 3, 4} and the connector genes and 29 of its 72 order keys from parent 2, the rest from parent 1.
- Child (before mutation): spines {0, 1, 2, 4}, 72 rooms, $F = 0.949$.

The child is better than both parents: the mix happened to combine a good corridor set with a workable room order.

Worked example: a crossover that hurts

- Parent 1: spines {0, 1, 2, 3, 4}, 63 rooms, $F = 0.902$.
- Parent 2: spines {0, 2, 3, 4}, 52 rooms, $F = 0.850$.
- The child takes spine blocks {1, 3} and 34 of its 72 order keys from parent 2, the rest from parent 1.
- Child (before mutation): spines {0, 2, 3, 4}, 51 rooms, $F = 0.793$.

The child is worse than both parents. Each parent's room order was tuned to its own corridors; put on

a different corridor set, the order no longer fits.

Worked example: what crossover achieved over a whole run

I logged every crossover in the example run and scored each child *before* mutation, next to its two parents (540 crossovers):

- 43 children (8%) were better than their better parent;
- 126 (23%) scored the same as their better parent;
- 220 (41%) fell between the two parents;
- 151 (28%) were worse than both.

On average the better parent scored 0.9486 and the child 0.9451. Crossover sometimes produces the jump that matters (the first example above), but most of the time it does not improve on what the parents already had. This matches the review comment that splicing two plans seldom makes architectural sense, and it is why the default search (Section 7) does without it.

6.5 Mutation

A child is changed in four ways:

- **Gaussian noise** of standard deviation σ_t on about 30% of the 44 corridor genes;
- **spine toggles**: each of the eleven blocks flips its on/off gene with probability 0.08;
- **key swaps**: with probability 0.45, 3 pairs of rooms exchange their order keys;
- **Gaussian noise** on about 10% of the order keys.

All genes are then clipped to $[0, 1]$. The strength σ_t falls linearly from 0.15 in the first generation to 0.03 in the last: wide exploration early, fine adjustment late.

Worked example: one mutation at full strength

Mutating the final best genome once with $\sigma = 0.15$ changed 13 of 44 corridor genes and 12 of 72 order keys. The active spines changed from {0, 1, 2, 4} to {1, 2, 4}, and the fitness went from 0.954 to 0.792. One toggled spine is enough to change the plan completely, which is why the best individual is always carried over unchanged (elitism) and why mutation gets weaker over the run.

6.6 The example run

Table 7: The GA on the running example (population 24, 30 generations, seed 7).

Generation	σ_t	Best F	Mean F of population	Rooms (best)	Layouts evaluated
1	0.150	0.9493	0.9049	72	50
2	0.146	0.9493	0.9032	72	76
3	0.142	0.9493	0.9261	72	102
5	0.133	0.9493	0.9296	72	154
10	0.113	0.9495	0.9246	72	284
15	0.092	0.9503	0.9275	72	414
20	0.071	0.9506	0.8984	72	544
25	0.051	0.9513	0.9325	72	674
30	0.030	0.9536	0.9221	72	804

The run evaluated 804 layouts: $P = 24$ to start, then in each of 30 generations $P - 2 = 22$ children and 4 local-search trials, $24 + 30 \times (22 + 4) = 804$. It ended with 72 of 72 rooms and $F = 0.9536$. The best layout was already close after the first generation (the seeds do most of the work); the rest of the run adds small gains.

7 Algorithm D: the systematic search (default)

With the architectural decoder the important decision is small: which of at most ten spines are on. A space that small can be searched in an orderly way, with no population and no crossover. The search has three stages and a fixed budget of $P \times G$ layouts, the same as the GA's.

7.1 Stage 1: enumerate the spine sets

With m candidate spines the first stage tries

- all spines switched on, and
- every set of one, two, three or four of the seven best spines,

each with no trim, no shift and all order keys at 0.5 (rooms in priority order). That is

$$1 + \sum_{k=1}^{4} \binom{\min(m,7)}{k} \;\le\; 1 + 7 + 21 + 35 + 35 = 99 \text{ layouts.}$$

Worked example: stage 1 on `fp1`

The plate has $m = 5$ candidates, so stage 1 evaluates $1 + 5 + 10 + 10 + 5 = 31$ spine sets.

Spines switched on	Rooms placed	Fitness F
{0, 1, 2, 3, 4}	72	0.9460
{0, 1}	72	0.9460
{0, 1, 2}	72	0.9460
{0, 1, 3}	72	0.9460
{0, 1, 4}	72	0.9460
{0, 1, 2, 3}	72	0.9460
…		
{2, 4}	15	0.7703
{3, 4}	9	0.7478
{4}	6	0.7267

Fitness ranges from 0.727 to 0.946. Two things stand out:

- **The corridor set decides almost everything.** Spine 4 alone places 6 rooms; any set containing spines 0 and 1 places all 72.
- **Different sets can give the same plan.** The best sets all contain spines 0 and 1 and have the same fitness, because the decoder drops the extra spines when they get no bays of their own (Step 4 of Section 4).

7.2 Stage 2: pattern search from the three best

The three best *distinct* layouts of stage 1 become starting points. For each one the search generates its *neighbours*, the layouts one small move away:

- switch one spine on or off (10 moves);
- for each active spine: trim the low end, trim the high end, shift it one way, shift it the other (4 moves each);
- redraw the connector elbows (1 move);
- move a few rooms earlier or later in the assignment order (4 moves, about one room in twelve each).

The neighbours are evaluated in a seeded random order, and the **first one that is better is accepted** ($F' > F + 10^{-6}$). The search then continues from the new layout. When no neighbour is better, that start is a *local optimum* and is left alone.

Worked example: one descent, move by move

Starting from the best stage-1 layout (spines {0, 1, 2, 3, 4}) and applying the rule above:

Step	Neighbours tried	Accepted move	F before	F after	Rooms
1	9	6 rooms moved later in the order	0.9460	0.9461	72
2	5	6 rooms moved earlier in the order	0.9461	0.9474	72
3	35	no neighbour is better: local optimum	0.9474	0.9474	72

The descent tried 49 neighbours in total. With 5 active spines a neighbourhood has $10+4\times5+1+4=35$ moves; the last row shows all of them tried without success.

7.3 Stage 3: iterated local search

If all three starts are local optima and budget remains, the search *kicks* the best layout: it applies two random moves at once, which is enough to leave the local optimum. It then descends again with the same pattern search and keeps the result only if it beats the best layout so far. This repeats until the budget is used.

7.4 The whole run on the example

Worked example: every improvement after stage 1

The real run made 720 decoder calls (695 distinct layouts; a genome seen before is not decoded again). After the 31 layouts of stage 1, the best fitness rose 3 times:

Decoder call	Move that produced it	F of its parent	F after
41	6 rooms moved earlier in the order	0.9460	0.9475
67	spine 1 trimmed at its high end	0.9475	0.9492
97	spine 1 trimmed at its high end	0.9475	0.9495

The run ended with 72 of 72 rooms and $F = 0.9495$. A second run with the same inputs returned exactly the same genome and fitness: yes.

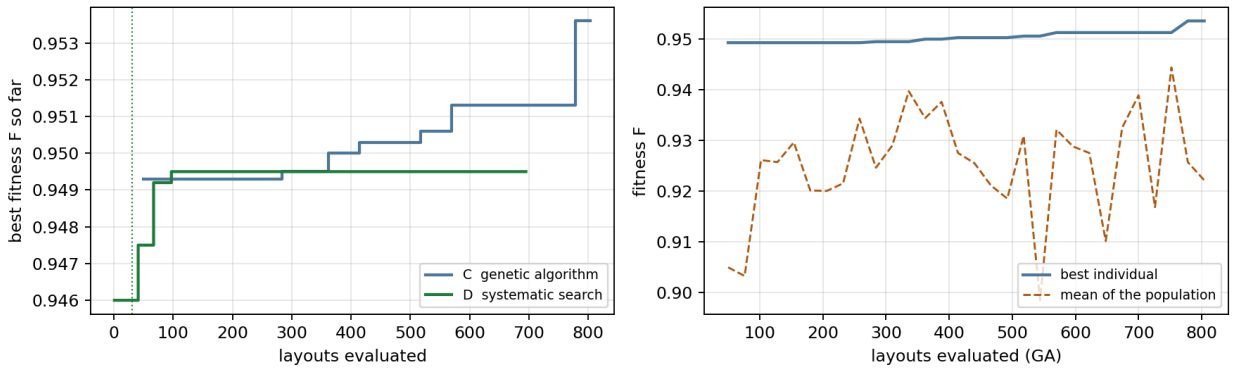


Figure 7: The two searches on the running example, same decoder and same budget. Left: best fitness so far against layouts evaluated; the dotted line marks the end of stage 1 of the systematic search. Right: the GA's best individual and the mean fitness of its population, which stays well below the best because mutation keeps breaking good plans.

Algorithm 3 Systematic search

Require: budget $B = P \times G$ layouts, seed
evaluate every spine set of stage 1 ▷ at most 99 layouts
starts ← the three best distinct layouts
while budget remains and some start is not a local optimum **do**
    for each start x that is not a local optimum **do**
        evaluate the neighbours of x in seeded random order; move x to the first better one
        if none is better, mark x as a local optimum
    end for
end while
while budget remains **do** ▷ iterated local search
    y ← best layout so far, changed by two random moves
    descend from y by the same pattern search
    if y is now better than the best layout, it becomes the best
end while
return the best layout

8 The algorithms side by side

Table 8: All four algorithms on the three sites, same program, same fitness (budget 24 × 30 for B, C and D, seed 7). "No window" counts guest rooms without a facade wall.

Site	Algorithm	Rooms	No window	F	Evaluated	Time	Checks
fp1	A reference GA	61/72	7	0.880	500 gen.	n/a	6/6
	B first attempt	53/72	3	0.880	804	119 s	5/6
	C updated GA	72/72	0	0.954	804	24 s	6/6
	D systematic	72/72	0	0.950	695	20 s	6/6
fp2	A reference GA	53/72	8	0.860	500 gen.	n/a	6/6
	B first attempt	60/72	8	0.893	804	102 s	6/6
	C updated GA	62/72	0	0.908	804	22 s	6/6
	D systematic	60/72	0	0.904	637	17 s	6/6
fp3	A reference GA	57/72	9	0.851	500 gen.	n/a	6/6
	B first attempt	61/72	3	0.894	804	115 s	6/6
	C updated GA	63/72	0	0.921	804	26 s	6/6
	D systematic	60/72	0	0.911	683	21 s	6/6

- **The decoder matters more than the search.** B has a better search and scoring than A but the same kind of decoder, and its plans keep the windowless rooms. C and D share the new decoder and neither has any.
- **GA or systematic?** At the same budget the GA (C) finishes slightly ahead on fitness: 0.928 against 0.922 on average, 197 rooms against 192. Its random order keys and crossover occasionally find a room order that the move-by-move search does not reach.
- **Why the systematic search is still the default.** It has no population size, crossover rate or mutation schedule to tune, it explains itself (every improvement is one named move), and it gives the same answer every time. The GA is one setting away in the Design Optimizer when the last few thousandths matter.

9 Algorithm E: exploring hotel layouts (shape, core and floors)

On the prototype's sites the plate and the cores are given. In the Hotel Schematic mode they are not: only the program and the buildable area are. An outer loop therefore *generates* plates and runs the decoder on each.

9.1 Sizing a plate

For K keys on F guest floors, let A be the net room area of the busiest level. With a slack factor $\varsigma = 1.04$, core area A_c and a packing efficiency $\eta = 0.93$, the plate must offer a corridor of length

$$R = \frac{\varsigma A + A_c}{2D\eta},$$

because every foot of double-loaded corridor serves $2D$ square feet of rooms. Each shape with bounding box $a \times b$ and wing depth $W_d = 2D + w$ offers

$$R_{\text{Bar}} = a, \quad R_{\text{L}} = R_{\text{T}} = a + b - W_d, \quad R_{\text{U}} = R_{\text{H}} = a + 2b - 2W_d, \quad R_{\text{Courtyard}} = 2a + 2b - 4W_d.$$

For each shape the generator finds the smallest a with $R_{\text{shape}}(a, \beta a) \geq R$ at aspect ratios $\beta \in \{0.6, 1\}$, in both orientations, and clips the result to the buildable area.

Worked example: plates for 117 keys on 4 guest floors

- Net area per level: 12,148, 10,995, 10,995, 10,995, 10,995 SF. The busiest is Level 1 with $A = 12,148$ SF.
- Elevators: $\max(2, \lceil 117/75 \rceil) = 2$. The main core is 32×26 ft and each stair 14×26 ft, so $A_c = 1,560$ SF.
- Room depth $D = 26$ ft and corridor $w = 6$ ft give a wing depth $W_d = 58$ ft.
- $R = (1.04 \times 12,148 + 1,560)/(2 \times 26 \times 0.93) = 293.5$ ft.

A Bar needs $a = 294$ ft, more than the 265 ft the buildable area allows, so it is clipped to 265 ft (the layout then has to do with less corridor than the estimate asks for). The generator produces 41 plates for this floor count:

Shape	$a \times b$, ft	Corridor run, ft	Footprint, SF	Facade, ft	Clipped	Core schemes
Bar	265 × 58	265	15,370	646	yes	center, ends
L	170 × 132	244	14,152	604	yes	center, ends, knuckle
L	220 × 132	294	17,052	704	no	center, ends, knuckle
L	170 × 176	288	16,704	692	yes	center, ends, knuckle
L	176 × 170	288	16,704	692	yes	center, ends, knuckle
U	170 × 112	278	16,124	672	yes	center, ends, knuckle
U	187 × 112	295	17,110	706	no	center, ends, knuckle
U	156 × 127	294	17,052	704	no	center, ends, knuckle
T	170 × 132	244	14,152	604	yes	center, ends, knuckle
T	220 × 132	294	17,052	704	no	center, ends, knuckle
T	170 × 176	288	16,704	692	yes	center, ends, knuckle
T	176 × 170	288	16,704	692	yes	center, ends, knuckle
H	156 × 136	312	18,096	740	no	center, ends, knuckle
Courtyard	156 × 156	392	22,736	784	no	center, ends, knuckle

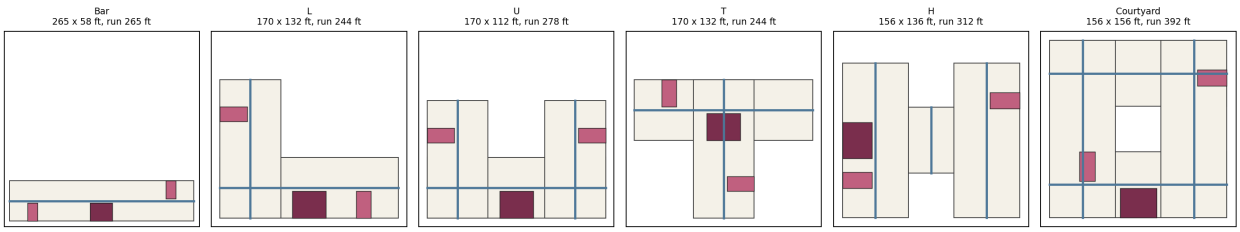


Figure 8: One generated plate per shape with the centre-core scheme. Dark: elevator core; lighter: stairs; blue: corridor centre-lines.

9.2 Cores and stairs

Each plate is tried with up to three core schemes:

- **ends**: the elevator core near one end, a stair near the other;
- **center**: the elevator core at the middle of the longest wing, and two stairs at the pair of wing ends that are farthest apart (which helps exit separation and keeps dead ends short);
- **knuckle**: the elevator core where two wings meet, with stairs at the far ends.

9.3 The three stages

1. **Screen.** Every plate, for every floor count tried (the requested count, one fewer, and up to two more, within the height limit), is decoded with 6 fixed spine sets (all spines, the best one, two, three and four, and every second one), rooms in priority order. Plates are independent, so they run in parallel processes.
2. **Refine.** The best plate of each shape, then the next best overall up to 8 plates, are improved by the pattern search of Section 7 for 24 evaluations each.
3. **Archive.** Every distinct layout is kept. Two layouts count as the same if they have the same plate, floor count, rooms placed, corridor area, shared-wall length and fitness (to three decimals).

One extra code check is added here because the floor count now varies: the number of stories (guest floors plus the ground level) must not exceed 5.

Worked example: the run on the Living + Lodging program

Buildable area 170 × 265 ft. The exploration screened 158 plate and floor-count variants, evaluated 1,140 layouts (158 × 6 in screening plus 8 × 24 in refinement) and kept 740 distinct ones in 36 s on eight processes. Refinement gains were small: Courtyard 156x156 · center core, 5 floors: 0.870 → 0.870; L 202x121 · ends core, 5 floors: 0.863 → 0.881; Bar 265x58 · center core, 5 floors: 0.863 → 0.863. The layout ranked first (next section) is Bar 265x58 · center core with 4 guest floors.

9.4 An upper bound per floor

Along a corridor of length R, both sides together offer $2R$ feet of frontage, less the frontage c_f taken by the cores. No layout can have more than

$$K_{\text{floor}} = \left\lfloor \frac{2R - c_f}{M} \right\rfloor$$

keys on a typical floor.

Worked example: bound for the Bar 265x58 · center core plate

$R = 265$ ft, the cores take $c_f = 84$ ft (the longer side of each of the 3 cores) and the module is $M = 13$ ft: $\lfloor (2 \times 265 - 84)/13 \rfloor = 34$ keys per floor, so at most $34 \times 4 = 136$ on 4 floors, against 117 requested. The bound is loose (it ignores suites, which take two bays, and the per-floor service rooms), but it shows at once whether a request can fit.

10 Choosing among the layouts: Pareto front and TOPSIS

The exploration returns hundreds of layouts. Which is best depends on what the owner values: the most keys, the lowest cost, the lowest building. That choice is kept separate from the fitness and is made in two deterministic steps over 12 criteria (fitness, keys, share of the program placed, daylight, corridor share, gross area per key, facade per key, height, dead end, exit travel, code checks passed, compactness).

10.1 Pareto front

Layout x *dominates* layout y if x is at least as good as y on every criterion and strictly better on at least one. A layout that nothing dominates is on the *Pareto front*. A dominated layout is never the right choice, whatever the priorities.

10.2 TOPSIS

TOPSIS ranks all layouts for a given set of weights λ_q:

1. **Normalise** each criterion q over the layouts j: $r_{jq} = x_{jq} / \sqrt{\sum_j x_{jq}^2}$.
2. **Weight**: $v_{jq} = \dfrac{\lambda_q}{\sum_q \lambda_q} r_{jq}$.
3. Find the **ideal** v_q^+ (the best value of each criterion over all layouts) and the **worst** v_q^-. "Best" is the largest value for a criterion to maximise and the smallest for one to minimise.
4. Measure each layout's **distance** to both: $d_j^{\pm} = \sqrt{\sum_q (v_{jq} - v_q^{\pm})^2}$.
5. The **closeness** $C_j = \dfrac{d_j^-}{d_j^+ + d_j^-}$ lies between 0 and 1. The layout with the largest C_j is ranked first.

Worked example: four real layouts, three criteria

To keep the arithmetic visible, take four layouts from the hotel run and only three of the criteria, with the weights of the Balanced preset: Fitness (all objectives) (maximise, $\lambda = 3$), Gross SF per key (minimise, $\lambda = 1$) and Dead-end corridor (minimise, $\lambda = 1$).

	Layout	Floors	Fitness	SF per key	Dead end, ft	On these three criteria
A	Bar 265x58 · center core	4	0.859	657	24.1	on the front
B	L 220x132 · knuckle core	4	0.849	729	26.1	dominated by A
C	Courtyard 156x156 · ends core	4	0.845	972	28.1	dominated by A
D	Bar 265x58 · center core	5	0.863	788	24.1	on the front

Layout B is dominated by A: A has higher fitness (0.859 against 0.849), less area per key (657 against 729 SF) and a dead end that is no longer (24.1 against 26.1 ft). Layouts A and D are both on the front: D has the higher fitness (0.863 against 0.859) but needs more area per key (788 against 657 SF), so neither beats the other on everything.

Normalisers $\sqrt{\sum_j x_{jq}^2}$ are 1.708, 1,589.9 and 51.3; the weights become 0.6, 0.2, 0.2 (each λ_q divided by 5).

	r_1	r_2	r_3	v_1	v_2	v_3	d^+	d^-	C
A	0.5030	0.4131	0.4697	0.3018	0.0826	0.0939	0.0012	0.0428	**0.974**
B	0.4972	0.4583	0.5087	0.2983	0.0917	0.1017	0.0128	0.0316	**0.711**
C	0.4947	0.6111	0.5477	0.2968	0.1222	0.1095	0.0430	0.0000	**0.000**
D	0.5050	0.4958	0.4697	0.3030	0.0992	0.0939	0.0165	0.0285	**0.633**

The ideal point is $(0.3030, 0.0826, 0.0939)$ and the worst $(0.2968, 0.1222, 0.1095)$. For layout A, $C = 0.0428/(0.0012 + 0.0428) = 0.974$. The ranking is A > B > D > C.

With all 12 criteria and all 740 layouts the same procedure produces the ranking shown in the app. The Pareto flags can differ from this small example, because a layout dominated on three criteria may be the best on a fourth.

10.3 Presets

The app offers five weight presets (Balanced, Most keys, Lowest cost, Guest experience, Low-rise) and sliders. Changing the weights re-ranks the stored layouts instantly; nothing is searched again. In the hotel run the Balanced preset ranks Bar 265x58 · center core first, while Most keys ranks Courtyard 156x156 · center core first.

11 How good can any algorithm be? Two upper bounds

Before judging a search it helps to know what is possible at all. Two bounds hold for every layout.

Area. A corridor of width w serves at most two room depths, so at least $w/(2D + w)$ of the used floor is corridor. With L levels of placeable area $|\Omega|$ and mean key area $\bar{a}$:

$$K \le K_{\text{area}} = \left\lfloor \frac{L\,|\Omega|\left(1 - \frac{w}{2D+w}\right)}{\bar{a}} \right\rfloor .$$

Windows. If every guest room needs a window, the rooms of one floor share the facade length P_f. With mean key frontage $\bar{f}$:

$$K \le K_{\text{window}} = L\left\lfloor \frac{P_f}{\bar{f}} \right\rfloor.$$

Worked example: bounds for `fp1`

$L = 3$, $|\Omega| = 9,640\,\text{SF}$, minimum corridor share 0.09, mean key area 327 SF: $K_{\text{area}} = \lfloor 3 \times 9,640 \times (1 - 0.09)/327 \rfloor = 80$. The facade is 489 ft per level and the mean key frontage 12.9 ft: $K_{\text{window}} = 3 \times \lfloor 489/12.9 \rfloor = 111$. The program asks for 69 keys, below both bounds, so the request is feasible in principle. The systematic search places 69 of them and the reference GA 58.

12 Summary

- **Representation first.** The reference GA and my first attempt searched over free corridor segments, a space in which most candidates are poor plans. The updated decoder restricts the search to plans with straight, facade-aligned corridors and modular bays. That single change removed every windowless guest room on all three sites.
- **Then a simple search is enough.** With at most ten spines to switch on or off, enumerating the combinations and improving the best ones move by move finds a layout within about 0.01 of the GA's fitness on every site, with the same answer on every run.
- **The GA is still useful.** It finishes slightly ahead at equal budget, and the logged run shows why: most crossovers do not help, but the few that do combine a corridor set and a room order that single moves reach slowly.
- **Score and choice are separate.** The fitness is fixed arithmetic. Which good layout to build is decided afterwards, by Pareto dominance and TOPSIS under weights the user controls.

A Parameters

Table 9: Main parameters and their default values.

Parameter	Value	Used by
Population P	24	C; with G it also sets the budget of D
Generations G	30	C; budget $P \times G$ for D
Elite	2	C: best individuals copied unchanged
Tournament size	3	C: selection
Crossover rate	0.9	C
Mutation σ	$0.15 \to 0.03$	C: linear schedule
Spine toggle rate	0.08	C: per block, per mutation
Local-search trials	4	C: per generation, on the best individual
Immigrants	2	C: only when the population has converged
Seed	7	all: fixes every random choice
On/off threshold	0.12	decoder: a spine is on at or above it
Maximum trim	35% per end	decoder
Maximum shift	±2 ft	decoder
Size tolerance	85%	decoder: minimum frontage and depth of a guest room
Remainder rule	$0.4M$	decoder: smaller remainders join the last bay
Starts in stage 2	3	D
Spine sets in stage 1	up to 99	D: sets of up to 4 of the best 7, plus all
Seeds per plate	6	E: screening
Plates refined	8	E: 24 evaluations each

B Reproducing this report

```
python src/build_optimizer_study.py                    # three-site comparison
python src/build_optimization_report.py --recompute  # worked examples + this PDF
python src/run_optimizer.py --method systematic      # one run of algorithm D
python src/run_optimizer.py --method ga              # one run of algorithm C
```

The code is in `src/rngd/optimize/`: `planner.py` (decoder), `metrics.py` (fitness), `search.py` (algorithms C and D), `engine.py` (the free-form decoder of B), `baseline.py` (loader for the reference results), `typology.py` and `explore.py` (algorithm E) and `decide.py` (Pareto and TOPSIS). The worked examples are produced by `src/optimization_examples.py`.